

Synopsis

NETWORK INTRUSION DETECTION FOR CYBER SECURITY

Problem Id:9

Mamta : 2028449

Shivangi : 2028495

Amrita sahuo : 2028637

Section:L

Problem description

In today's highly connected digital environment, network systems are continuously exposed to a wide range of cyber threats, including unauthorized access, data breaches, denial-of-service attacks, and other malicious activities. As the volume and complexity of network traffic increase, traditional rule-based security systems struggle to effectively identify and respond to new or evolving attack patterns. This creates a critical need for intelligent and automated solutions that can analyze large-scale network data and detect intrusions accurately and efficiently. The problem addressed in this project is the development of a Network Intrusion Detection System (NIDS) capable of distinguishing between normal and malicious network behavior using a data-driven approach.

To solve this problem, the project utilizes raw and unprocessed network traffic data obtained from publicly available sources such as Kaggle. This data is often noisy, incomplete, and contains a mixture of numerical and categorical features, making it unsuitable for direct use in machine learning models. Therefore, a crucial part of the problem involves performing comprehensive data preprocessing, including handling missing values, removing duplicates, encoding categorical variables, and scaling numerical features. These steps ensure that the dataset becomes structured, consistent, and suitable for analysis. Once the data is properly preprocessed, a machine learning algorithm, specifically K-Nearest Neighbors (KNN), is applied to classify network connections based on similarity measures such as distance metrics.

The core objective is to build a system that can learn from historical network data and accurately predict whether a new network instance represents normal activity or a potential intrusion. This involves addressing several challenges, such as selecting the optimal number of neighbors (K) in KNN, managing imbalanced datasets where attack instances may be less frequent than normal ones, and ensuring that the model performs efficiently on large datasets. Additionally, the system must minimize false positives (normal traffic incorrectly flagged as attacks) and false negatives (attacks going undetected), as both can have serious consequences in real-world applications.

Ultimately, the problem focuses on designing a scalable and reliable intrusion detection mechanism that leverages machine learning techniques to enhance network security. The expected outcome is a robust model capable of analyzing network traffic with high accuracy, supported by a well-preprocessed dataset, and adaptable to real-world scenarios where timely and precise detection of cyber threats is essential.

Steps of implementation

Data collection

The first step in the implementation is data collection, where a suitable dataset for network intrusion detection is obtained from reliable sources such as Kaggle. These datasets typically contain raw network traffic records with multiple features describing each connection, including attributes like duration, protocol type, service, and error rates. The data is often available in compressed formats and may require extraction and conversion into a structured format such as CSV. It is important to work with raw and unprocessed data at this stage so that all preprocessing steps can be applied systematically according to the project requirements. Proper data collection ensures that the dataset is comprehensive and representative of both normal and malicious network activities.

Data preprocessing

Data preprocessing is a critical stage where the raw dataset is cleaned and transformed into a suitable format for machine learning. This step involved several steps -

Data Loading

The dataset is loaded into the system using appropriate tools such as Pandas. This step ensures that the data is available in a structured format for further processing.

Handling Missing Values

The dataset is checked for missing or null values. These values are handled by replacing them with suitable measures such as mean (for numerical data) or mode (for categorical data) to maintain data consistency.

Removing Duplicate Records

Duplicate entries in the dataset are identified and removed. This helps in reducing redundancy and prevents bias in the model during training.

Encoding Categorical Data

Categorical features such as protocol type, service, and flags are converted into numerical form using encoding techniques like One-Hot Encoding or Label Encoding, as machine learning models require numerical input.

Feature Selection / Separation

Relevant features are selected from the dataset, and the data is divided into input features (X) and target variable (y). This helps in preparing the dataset for model training.

Feature Scaling (Normalization/Standardization)

Numerical features are scaled using techniques like StandardScaler to ensure that all features have equal importance. This is especially important for distance-based algorithms like KNN.

Data Transformation

The processed data is converted into a suitable format (such as NumPy arrays) required by machine learning algorithms for efficient computation.

Model selection

In this step, an appropriate machine learning algorithm is chosen for the problem. For this project, the K-Nearest Neighbors (KNN) algorithm is selected due to its simplicity and effectiveness in classification tasks. KNN works by classifying a data point based on the majority class of its nearest neighbors using distance metrics such as Euclidean distance. The selection of this model is based on its ability to handle multi-class classification problems and its suitability for pattern recognition in network traffic data. Choosing the right model is essential for achieving accurate and reliable results.

Training and validation

After selecting the model, the dataset is divided into training and validation (or testing) sets. The training set is used to train the KNN model by allowing it to learn patterns from the data. During this phase, different values of K (number of neighbors) are tested to determine the optimal configuration. Validation techniques are used to evaluate the model's performance during training and to prevent overfitting. This step ensures that the model generalizes well and performs effectively not only on the training data but also on unseen data.

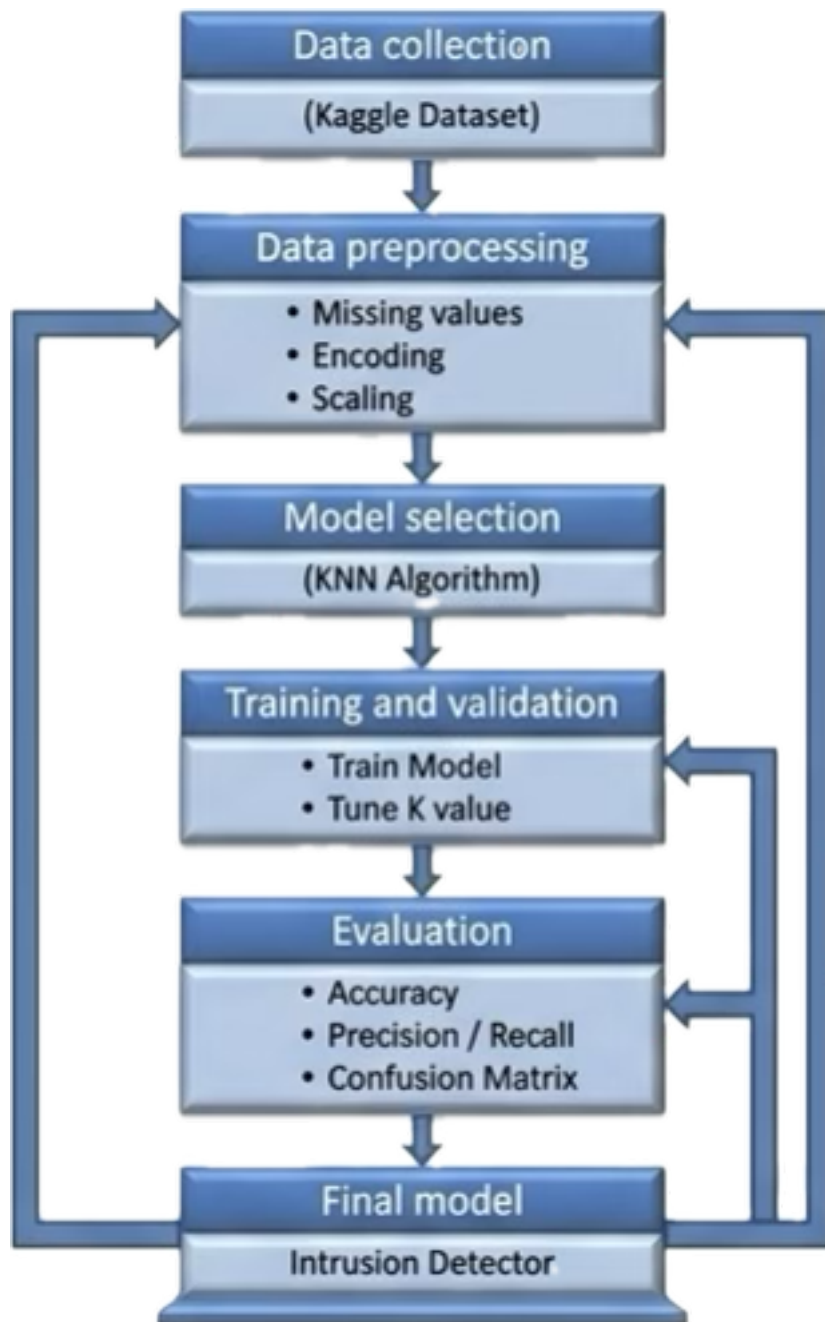
Evaluation

Once the model is trained, it is evaluated using the validation or testing dataset. Various performance metrics such as accuracy, precision, recall, and confusion matrix are used to assess how well the model classifies network traffic. Accuracy provides an overall measure of correctness, while precision and recall help in understanding the model's effectiveness in detecting intrusions and minimizing false alarms. The evaluation step is crucial for identifying any weaknesses in the model and determining whether further tuning or improvements are required.

Final model

In the final step, the best-performing model is selected based on the evaluation results. This

model is then used to make predictions on new or unseen network data, enabling the system to classify activities as normal or malicious. The final model can be further optimized and integrated into real-world applications for real-time intrusion detection. The goal is to develop a reliable and scalable system that enhances network security by accurately identifying potential threats.



Expected modules / libraries to be used

Pandas

Pandas is used to load the network intrusion dataset and perform data cleaning tasks like handling missing values and removing duplicates. It is also used to prepare features and

labels for the model.

NumPy

NumPy is used to convert the processed data into arrays for efficient computation. These arrays are then used as input for training the KNN model.

Matplotlib

Matplotlib is used to plot graphs such as accuracy vs K values. This helps in selecting the best value of K for the KNN model.

Seaborn

Seaborn is used to plot the confusion matrix after model prediction. It helps visualize correct and incorrect classifications.

Scikit-learn (sklearn)

Scikit-learn is used to implement the full ML pipeline including preprocessing, model training, and evaluation. It provides ready-to-use functions for KNN and metrics.

StandardScaler

StandardScaler is used to scale the feature values before applying KNN. This ensures that all features contribute equally in distance calculation.

LabelEncoder / OneHotEncoder

These are used to convert categorical features like protocol type into numerical form. This allows the KNN model to process the data.

KNeighborsClassifier

KNeighborsClassifier is used to train the KNN model on the dataset. It predicts whether a network connection is normal or an intrusion.

Train-Test Split

This is used to split the dataset into training and testing data. The model is trained on one part and tested on unseen data.

Metrics

Metrics are used after prediction to evaluate model performance. They measure how accurately the model detects intrusions.

Expected evaluation metrics and plots to be used

Accuracy

Accuracy measures the overall correctness of the model by calculating the ratio of correctly predicted instances to total instances. In this project, it helps determine how well the model classifies network traffic as normal or attack.

Precision

Precision measures how many of the predicted attack instances are actually correct. It is important in this project to reduce false alarms (false positives) in intrusion detection.

Recall (Detection Rate)

Recall measures how many actual attacks are correctly identified by the model. It is very important in intrusion detection because missing an attack (false negative) can be dangerous.

F1-Score

F1-score is the harmonic mean of precision and recall. It provides a balanced measure when both false positives and false negatives are important.

Confusion Matrix

A confusion matrix shows the number of correct and incorrect predictions in terms of true positives, true negatives, false positives, and false negatives. It gives a complete picture of model performance.

Accuracy vs K Plot (Elbow Method)

This plot shows how accuracy changes with different values of K in the KNN algorithm. It helps in selecting the optimal value of K for best performance.

Feature Distribution Plots

These plots show how different features are distributed in the dataset. They help in understanding the data before and after preprocessing.

Platform required

Jupyter notebook

We used Jupyter notebook for implementation because it allows step by step execution and easy visualisation of data and results.

Kaggle

To download the dataset

Python

For implementation of code

GitHub

For project submission

Books and link sources

Network Intrusion Detection System Using Machine Learning.

https://indjst.org/articles/network-intrusion-detection-system-using-machine-learning?utm_source=chatgpt.com

Kaggle (Dataset Source).

<https://www.kaggle.com/datasets/programmer3/nsl-kdd-intrusion-detection-dataset>

Machine learning for network intrusion detection.

<https://www.geeksforgeeks.org/machine-learning/intrusion-detection-system-using-machine-learning-algorithms/>